Activity 2.1 Develop the core functionalities:

Generate Structured Comic Panel Outline

Function: generate_outline()
File: gemini_flash.py

- This function uses the **Gemini Flash** AI model to generate a structured 5-panel comic outline based on the user's story prompt.
- Each panel contains a panel number, title, scene description, and an image generation prompt.
- The output is a list of dictionaries, one for each comic panel.

```
def generate_outline(user_prompt: str) -> list:
    """
    Generates a 5-panel comic layout based on the user's story idea using Gemini.

    Args:
        user_prompt (str): The user's comic idea prompt.

    Returns:
        list: A list of dictionaries, one for each panel.
    """
    prompt = f"""
You are a professional AI comic planner.

Your task is to generate a *strictly formatted* JSON array containing 5 panel descriptions for a comic based on the story idea below:

STORY: "{user_prompt}"

Each JSON object must include:
- "panel" (integer)
- "title" (string)
- "scene_description" (string)
- "image_prompt" (string)

Respond ONLY in this valid JSON format, without any explanations or markdown:
[
  {{
    "panel": 1,
    "title": "Title here",
    "scene_description": "Scene description here",
    "image_prompt": "Image prompt for Stable Diffusion"
  }},
  ...
]
"""
```

```
try:
    response = model.generate_content(prompt)
    output_text = response.text.strip()

    print("\n🔥 RAW GEMINI RESPONSE 🔥\n", output_text)

    # Remove any markdown formatting if present
    if output_text.startswith("```json"):
        output_text = output_text.replace("```json", "").replace("```", "").strip()

    panel_data = json.loads(output_text)

    # Additional structure validation
    if not isinstance(panel_data, list):
        raise ValueError("Gemini response is not a list.")

    for panel in panel_data:
        if not isinstance(panel, dict) or not all(key in panel for key in ("panel", "title", "scene_description", "image_prompt")):
            raise ValueError(f"Invalid panel format or missing keys: {panel}")

    return panel_data

except json.JSONDecodeError as e:
    print("❌ JSON Decode Error:", e)
    print("❌ Full Text Received:\n", output_text)
    return [{"error": f"JSON parsing failed: {str(e)}"}]

except Exception as e:
    print("❌ Unexpected Error:", e)
    return [{"error": f"Generation failed: {str(e)}"}]
```

Generate Comic Story Narration and Dialogue

Function: generate_story()
File: gemini_pro.py

- This function uses the **Gemini Pro** AI model to expand the panel outlines into a full comic-style story.
- It creates engaging narration and character dialogues for each panel.
- The output is a single formatted text containing all the panels' stories.

```python
def generate_story(outline: list) -> str:
    """
    Generates a detailed comic story with narration and character dialogue
    from a list of comic panel outlines using Gemini 1.5 Pro.

    Args:
        outline (list): A list of strings representing each comic panel's idea.

    Returns:
        str: The generated comic story text or an error message.
    """

    # Format the panel outline as a numbered list for clarity
    formatted_outline = "\n".join([f"{i+1}. {item}" for i, item in enumerate(outline)])

    # Construct the prompt
    prompt = f"""
You're a comic book writer.

Given the following panel breakdown, write a comic-style story with engaging narration and character dialogues for each panel.

Panel Outline:
{formatted_outline}

Guidelines:
- Use a fun and engaging tone, like an actual comic book.
- Include narration and clearly marked character lines.
- Keep each panel self-contained but part of a cohesive story.
"""

    try:
        response = model.generate_content(prompt)
        return response.text
    except Exception as e:
        return f"Error generating story: {str(e)}"
```

Generate Comic Illustrations from Prompts

Function: generate_image()
File: image_generator.py

- This function uses the **Stable Diffusion** model to create a comic-style image based on the provided image prompt.
- It sanitizes the prompt for safe filenames and saves the generated image to the static/panels directory.
- Returns the file path to the saved image.